```
!pip install -q xgboost imbalanced-learn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from imblearn.over_sampling import SMOTE

import warnings
warnings.filterwarnings("ignore")
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
df = pd.read_csv("pcos_prediction_dataset_.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

```
Dataset loaded successfully!
Shape: (240000, 17)
```

```
df.head()
```

	Country	Age	BMI	Menstrual Regularity	Hirsutism	Acne Severity	Family History of PCOS	Insulin Resistance	Lifestyle Score	Stress Levels	Urban/Rural	Socioecono Sta
0	Madagascar	26.0	Overweight	Regular	Yes	Severe	Yes	Yes	2.0	Low	Rural	H
1	Vietnam	16.0	Underweight	Regular	Yes	NaN	No	Yes	4.0	High	Rural	Mi
2	Somalia	41.0	Normal	Regular	No	Moderate	No	No	7.0	Medium	Urban	Mi
3	Malawi	27.0	Normal	Irregular	No	Mild	No	No	10.0	Low	Urban	H
4	France	26.0	Overweight	Irregular	Yes	NaN	No	No	7.0	Medium	Urban	Mi

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\nColumns:")
for col in df.columns:
    print(col)
```

```
Number of rows: 240000
Number of columns: 17

Columns:
Country
Age
BMI
Menstrual Regularity
Hirsutism
Acne Severity
Family History of PCOS
Insulin Resistance
Lifestyle Score
Stress Levels
Urban/Rural
```

```
Socioeconomic Status
Awareness of PCOS
Fertility Concerns
Undiagnosed PCOS Likelihood
Ethnicity
Diagnosis
```

```
print(df.dtypes)
```

```
Country                         object
Age                            float64
BMI                             object
Menstrual Regularity            object
Hirsutism                       object
Acne Severity                   object
Family History of PCOS          object
Insulin Resistance              object
Lifestyle Score                float64
Stress Levels                   object
Urban/Rural                     object
Socioeconomic Status            object
Awareness of PCOS               object
Fertility Concerns              object
Undiagnosed PCOS Likelihood    float64
Ethnicity                       object
Diagnosis                       object
dtype: object
```

```
missing = df.isnull().sum()

print("Missing values:")
print(missing[missing > 0])
```

```
Missing values:
Acne Severity    60085
dtype: int64
```

```
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

```
print("Diagnosis distribution:")
print(df["Diagnosis"].value_counts())

print("\nDiagnosis percentage:")
print(
    df["Diagnosis"].value_counts(normalize=True) * 100
)
```

```
Diagnosis distribution:
Diagnosis
No     147411
Yes     92589
Name: count, dtype: int64

Diagnosis percentage:
Diagnosis
No     61.42125
Yes    38.57875
Name: proportion, dtype: float64
```

```
X = df.drop(columns=["Diagnosis"]).copy()
y = df["Diagnosis"].copy()

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (240000, 16)
y shape: (240000,)
```

```
y = y.map({
    "No": 0,
    "Yes": 1
})

print(y.value_counts())

print("\nMissing target values:", y.isnull().sum())
```

```
Diagnosis
0    147411
1     92589
Name: count, dtype: int64

Missing target values: 0
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (168000, 16)
Testing data: (72000, 16)
```

```
numeric_cols = X_train.select_dtypes(
    include=["int64", "float64", "int32", "float32"]
).columns.tolist()

categorical_cols = X_train.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

print("Numerical columns:")
print(numeric_cols)

print("\nCategorical columns:")
print(categorical_cols)
```

```
Numerical columns:
['Age', 'Lifestyle Score', 'Undiagnosed PCOS Likelihood']

Categorical columns:
['Country', 'BMI', 'Menstrual Regularity', 'Hirsutism', 'Acne Severity', 'Family History of PCOS', 'Insulin Resistance', 'Stress
```

```
for col in numeric_cols:
    median_value = X_train[col].median()

    X_train[col] = X_train[col].fillna(median_value)
    X_test[col] = X_test[col].fillna(median_value)
```

```
for col in categorical_cols:
    mode_value = X_train[col].mode()[0]

    X_train[col] = X_train[col].fillna(mode_value)
    X_test[col] = X_test[col].fillna(mode_value)
```

```
X_train_encoded = pd.get_dummies(
    X_train,
    columns=categorical_cols,
    drop_first=True
)

X_test_encoded = pd.get_dummies(
    X_test,
    columns=categorical_cols,
    drop_first=True
)
```

```
X_train_encoded, X_test_encoded = X_train_encoded.align(
    X_test_encoded,
    join="left",
    axis=1,
    fill_value=0
)

print("Training features:", X_train_encoded.shape)
print("Testing features:", X_test_encoded.shape)
```

```
Training features: (168000, 99)
Testing features: (72000, 99)
```

```
X_train_encoded = X_train_encoded.astype(float)
X_test_encoded = X_test_encoded.astype(float)

print(
    X_train_encoded.dtypes.value_counts()
)
```

```
float64    99
Name: count, dtype: int64
```

```
print("NaN in X_train:",
      X_train_encoded.isna().sum().sum())

print("NaN in X_test:",
      X_test_encoded.isna().sum().sum())

print("Infinity in X_train:",
      np.isinf(X_train_encoded).sum().sum())

print("Infinity in X_test:",
      np.isinf(X_test_encoded).sum().sum())
```

```
NaN in X_train: 0
NaN in X_test: 0
Infinity in X_train: 0
Infinity in X_test: 0
```

```
print("Before SMOTE:")
print(y_train.value_counts())

print("\nPercentage:")
print(
    y_train.value_counts(normalize=True) * 100
)
```

```
Before SMOTE:
Diagnosis
0    103188
1     64812
Name: count, dtype: int64

Percentage:
Diagnosis
0    61.421429
1    38.578571
Name: proportion, dtype: float64
```

```
smote = SMOTE(
    random_state=42
)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train_encoded,
    y_train
)

print("Before SMOTE:")
print(y_train.value_counts())

print("\nAfter SMOTE:")
print(y_train_smote.value_counts())
```

```
Before SMOTE:
Diagnosis
```

```
0    103188
1     64812
Name: count, dtype: int64

After SMOTE:
Diagnosis
0    103188
1    103188
Name: count, dtype: int64
```

```
lr = LogisticRegression(
    max_iter=1000,
    random_state=42
)

lr.fit(
    X_train_smote,
    y_train_smote
)

lr_pred = lr.predict(X_test_encoded)

lr_prob = lr.predict_proba(
    X_test_encoded
)[:, 1]

print("Logistic Regression completed!")
```

```
Logistic Regression completed!
```

```
rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=-1,
    class_weight=None
)

rf.fit(
    X_train_smote,
    y_train_smote
)

rf_pred = rf.predict(X_test_encoded)

rf_prob = rf.predict_proba(
    X_test_encoded
)[:, 1]

print("Random Forest completed!")
```

```
Random Forest completed!
```

```
xgb = XGBClassifier(
    objective="binary:logistic",
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1,
    tree_method="hist"
)

print("XGBoost initialized!")
```

```
XGBoost initialized!
```

```
param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [2, 3, 4, 5, 6],
    "learning_rate": [0.01, 0.03, 0.05, 0.1],
    "subsample": [0.7, 0.8, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.8, 0.9, 1.0],
    "min_child_weight": [1, 3, 5],
    "gamma": [0, 0.1, 0.3]
}

random_search = RandomizedSearchCV(
    estimator=xgb,
```

```
    param_distributions=param_grid,
    n_iter=15,
    scoring="f1",
    cv=3,
    random_state=42,
    n_jobs=-1,
    verbose=1
)

random_search.fit(
    X_train_smote,
    y_train_smote
)

print("XGBoost tuning completed!")
```

```
Fitting 3 folds for each of 15 candidates, totalling 45 fits
XGBoost tuning completed!
```

```
best_xgb = random_search.best_estimator_

print("Best XGBoost Parameters:")
print(random_search.best_params_)
```

```
Best XGBoost Parameters:
{'subsample': 0.9, 'n_estimators': 300, 'min_child_weight': 1, 'max_depth': 6, 'learning_rate': 0.1, 'gamma': 0.1, 'colsample_by
```

```
xgb_pred = best_xgb.predict(
    X_test_encoded
)

xgb_prob = best_xgb.predict_proba(
    X_test_encoded
)[:, 1]

print("Predictions completed!")
```

```
Predictions completed!
```

```
results = pd.DataFrame({

    "Model": [
        "Logistic Regression",
        "Random Forest",
        "Tuned XGBoost"
    ],

    "Accuracy": [
        accuracy_score(y_test, lr_pred),
        accuracy_score(y_test, rf_pred),
        accuracy_score(y_test, xgb_pred)
    ],

    "Precision": [
        precision_score(y_test, lr_pred, zero_division=0),
        precision_score(y_test, rf_pred, zero_division=0),
        precision_score(y_test, xgb_pred, zero_division=0)
    ],

    "Recall": [
        recall_score(y_test, lr_pred, zero_division=0),
        recall_score(y_test, rf_pred, zero_division=0),
        recall_score(y_test, xgb_pred, zero_division=0)
    ],

    "F1 Score": [
        f1_score(y_test, lr_pred, zero_division=0),
        f1_score(y_test, rf_pred, zero_division=0),
        f1_score(y_test, xgb_pred, zero_division=0)
    ],

    "ROC-AUC": [
        roc_auc_score(y_test, lr_prob),
        roc_auc_score(y_test, rf_prob),
        roc_auc_score(y_test, xgb_prob)
```

```
    ]
})

results = results.round(4)

display(results)
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression	0.9042	0.9027	0.8426	0.8716	0.9177
1	Random Forest	0.9353	0.9710	0.8578	0.9109	0.9321
2	Tuned XGBoost	0.9353	0.9832	0.8466	0.9098	0.9335

```
best_model = results.loc[
    results["F1 Score"].idxmax()
]

print("Best Model Based on F1 Score:")
print(best_model)
```

```
Best Model Based on F1 Score:
Model        Random Forest
Accuracy            0.9353
Precision            0.971
Recall              0.8578
F1 Score            0.9109
ROC-AUC             0.9321
Name: 1, dtype: object
```

```
print(
    classification_report(
        y_test,
        xgb_pred,
        target_names=[
            "No PCOS",
            "PCOS"
        ],
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

     No PCOS       0.91      0.99      0.95     44223
        PCOS       0.98      0.85      0.91     27777

    accuracy                           0.94     72000
   macro avg       0.95      0.92      0.93     72000
weighted avg       0.94      0.94      0.93     72000
```

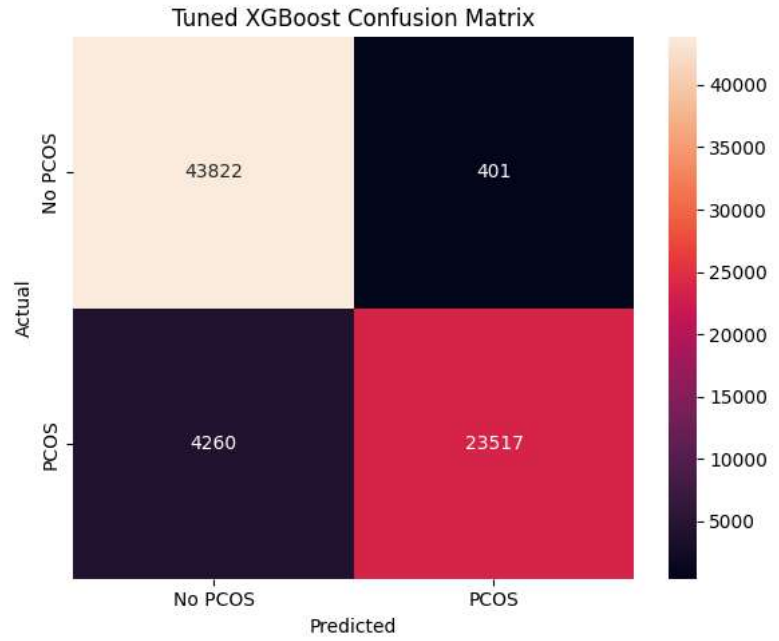
```
cm = confusion_matrix(
    y_test,
    xgb_pred
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=["No PCOS", "PCOS"],
    yticklabels=["No PCOS", "PCOS"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Tuned XGBoost Confusion Matrix")

plt.tight_layout()
plt.show()
```




```
thresholds = np.arange(
    0.10,
    0.91,
    0.05
)

threshold_results = []

for threshold in thresholds:

    prediction = (
        xgb_prob >= threshold
    ).astype(int)

    threshold_results.append({

        "Threshold": round(threshold, 2),

        "Accuracy": accuracy_score(
            y_test,
            prediction
        ),

        "Precision": precision_score(
            y_test,
            prediction,
            zero_division=0
        ),

        "Recall": recall_score(
            y_test,
            prediction,
            zero_division=0
        ),

        "F1 Score": f1_score(
            y_test,
            prediction,
            zero_division=0
        )
    })

threshold_results = pd.DataFrame(
    threshold_results
)

display(threshold_results)
```

	Threshold	Accuracy	Precision	Recall	F1 Score
0	0.10	0.729889	0.597609	0.917918	0.723915
1	0.15	0.904000	0.883139	0.865716	0.874341
2	0.20	0.925778	0.944130	0.858408	0.899231
3	0.25	0.930833	0.961198	0.855240	0.905128
4	0.30	0.932389	0.968276	0.852684	0.906811
5	0.35	0.933375	0.972723	0.851172	0.907897
6	0.40	0.934000	0.976294	0.849552	0.908524
7	0.45	0.934806	0.980076	0.848256	0.909414
8	0.50	0.935264	0.983234	0.846636	0.909837
9	0.55	0.935153	0.985299	0.844512	0.909489
10	0.60	0.934944	0.987544	0.841992	0.908978
11	0.65	0.934222	0.988674	0.839111	0.907774
12	0.70	0.933514	0.990359	0.835799	0.906539
13	0.75	0.932694	0.992737	0.831623	0.905066
14	0.80	0.930722	0.993931	0.825467	0.901900
15	0.85	0.927458	0.995213	0.815891	0.896675
16	0.90	0.921875	0.996503	0.800302	0.887691

```
best_threshold = threshold_results.loc[
    threshold_results["F1 Score"].idxmax()
]

print("Best Threshold:")
print(best_threshold)
```

```
Best Threshold:
Threshold    0.500000
Accuracy     0.935264
Precision    0.983234
Recall       0.846636
F1 Score     0.909837
Name: 8, dtype: float64
```

```
optimal_threshold = float(
    best_threshold["Threshold"]
)

final_prediction = (
    xgb_prob >= optimal_threshold
).astype(int)

print(
    "Optimal Threshold:",
    optimal_threshold
)
```

```
Optimal Threshold: 0.5
```

```
final_accuracy = accuracy_score(
    y_test,
    final_prediction
)

final_precision = precision_score(
    y_test,
    final_prediction,
    zero_division=0
)

final_recall = recall_score(
    y_test,
    final_prediction,
    zero_division=0
```

```
)

final_f1 = f1_score(
    y_test,
    final_prediction,
    zero_division=0
)

final_auc = roc_auc_score(
    y_test,
    xgb_prob
)

print("FINAL XGBOOST PERFORMANCE")
print("-------------------------")
print("Accuracy :", round(final_accuracy, 4))
print("Precision:", round(final_precision, 4))
print("Recall   :", round(final_recall, 4))
print("F1 Score :", round(final_f1, 4))
print("ROC-AUC  :", round(final_auc, 4))
```

```
FINAL XGBOOST PERFORMANCE
-------------------------
Accuracy : 0.9353
Precision: 0.9832
Recall   : 0.8466
F1 Score : 0.9098
ROC-AUC  : 0.9335
```

```
final_accuracy = accuracy_score(
    y_test,
    final_prediction
)

final_precision = precision_score(
    y_test,
    final_prediction,
    zero_division=0
)

final_recall = recall_score(
    y_test,
    final_prediction,
    zero_division=0
)

final_f1 = f1_score(
    y_test,
    final_prediction,
    zero_division=0
)

final_auc = roc_auc_score(
    y_test,
    xgb_prob
)

print("FINAL XGBOOST PERFORMANCE")
print("-------------------------")
print("Accuracy :", round(final_accuracy, 4))
print("Precision:", round(final_precision, 4))
print("Recall   :", round(final_recall, 4))
print("F1 Score :", round(final_f1, 4))
print("ROC-AUC  :", round(final_auc, 4))
```

```
FINAL XGBOOST PERFORMANCE
-------------------------
Accuracy : 0.9353
Precision: 0.9832
Recall   : 0.8466
F1 Score : 0.9098
ROC-AUC  : 0.9335
```

```
risk_category = pd.cut(
    xgb_prob,

    bins=[
```

```
        0,
        0.30,
        0.70,
        1.00
    ],

    labels=[
        "Low Risk",
        "Medium Risk",
        "High Risk"
    ],

    include_lowest=True
)

print(
    risk_category.value_counts()
)
```

```
Low Risk       47539
Medium Risk     1019
High Risk      23442
Name: count, dtype: int64
```

```
Tableau_data = df.loc[
    X_test.index
].copy()

print(
    "Tableau Dataset Shape:",
    Tableau_data.shape
)
```

```
Tableau Dataset Shape: (72000, 17)
```

```
Tableau_data["Actual_Diagnosis"] = y_test.values

Tableau_data["Predicted_Diagnosis"] = final_prediction

Tableau_data["PCOS_Probability"] = xgb_prob

Tableau_data["Risk_Category"] = (
    risk_category.astype(str)
)

Tableau_data["Model"] = "Tuned XGBoost"
```

```
Tableau_data["Actual_Diagnosis"] = (
    Tableau_data["Actual_Diagnosis"]
    .map({
        0: "No",
        1: "Yes"
    })
)

Tableau_data["Predicted_Diagnosis"] = (
    Tableau_data["Predicted_Diagnosis"]
    .map({
        0: "No",
        1: "Yes"
    })
)
```

```
Tableau_data.head()
```

	Country	Age	BMI	Menstrual Regularity	Hirsutism	Acne Severity	Family History of PCOS	Insulin Resistance	Lifestyle Score	Stress Levels	...	Awareness of PCOS	Fertilit Concern
41814	Taiwan	17.0	Normal	Regular	Yes	Moderate	Yes	No	10.0	Medium	...	No	N
47214	South Africa	36.0	Normal	Irregular	No	Mild	No	Yes	8.0	High	...	No	N
	Kingdom												

5 rows × 22 columns

```
print("Final Columns:")

print(
    Tableau_data.columns.tolist()
)
```

```
Final Columns:
['Country', 'Age', 'BMI', 'Menstrual Regularity', 'Hirsutism', 'Acne Severity', 'Family History of PCOS', 'Insulin Resistance',
```

```
Tableau_data.to_csv(
    "PCOS_Tuned_XGBoost_Predictions.csv",
    index=False
)

print(
    "Tableau prediction dataset created successfully!"
)
```

```
Tableau prediction dataset created successfully!
```

```
results.to_csv(
    "PCOS_Model_Comparison.csv",
    index=False
)

print(
    "Model comparison file created successfully!"
)
```

```
Model comparison file created successfully!
```

```
!ls -lh /content/*.csv
```

```
-rw-r--r-- 1 root root 201 Aug 29 06:48 /content/PCOS_Model_Comparison.csv
-rw-r--r-- 1 root root 26M Aug 29 06:28 /content/pcos_prediction_dataset_.csv
-rw-r--r-- 1 root root 11M Aug 29 06:48 /content/PCOS_Tuned_XGBoost_Predictions.csv
```

```
Start coding or generate with AI.
```